

INTERFACE IN DART

Introduction

In this section, you will learn the dart interface and how to implement an interface with the help of examples. In Dart, every class is **implicit interface**. Before learning about the interface in dart, you should have a basic understanding of the [class and objects](#), [inheritance](#) and [abstract class](#) in Dart.

Interface In Dart

An interface defines a syntax that a class must follow. It is a contract that defines the capabilities of a class. It is used to achieve abstraction in the Dart programming language. When you implement an interface, you must implement all the properties and methods defined in the interface. Keyword **implements** is used to implement an interface.

Syntax Of Interface In Dart

```
class InterfaceName {  
    // code  
}  
  
class ClassName implements InterfaceName {  
    // code  
}
```

Declaring Interface In Dart

In dart there is no keyword **interface** but you can use **class** or **abstract class** to declare an interface. All classes implicitly define an interface. Mostly **abstract class** is used to declare an interface.

```
// creating an interface using abstract class  
abstract class Person {  
    canWalk();  
}
```

```
canRun();  
}
```

Implementing Interface In Dart

You must use the **implements** keyword to implement an interface. The class that implements an interface must implement all the methods and properties of the interface.

```
class Student implements Person {  
  // implementation of canWalk()  
  @override  
  canWalk() {  
    print('Student can walk');  
  }  
  
  // implementation of canRun()  
  @override  
  canRun() {  
    print('Student can run');  
  }  
}
```

Example 1: Interface In Dart

In this example below, there is an interface **Laptop** with two methods **turnOn()** and **turnOff()**. The class **MacBook** implements the interface and overrides the methods to print the message.

```
// creating an interface using concrete class  
class Laptop {  
  // method  
  turnOn() {  
    print('Laptop turned on');  
  }  
  // method  
  turnOff() {  
    print('Laptop turned off');  
  }  
}  
  
class MacBook implements Laptop {  
  // implementation of turnOn()  
  @override  
  turnOn() {  
    print('MacBook turned on');  
  }  
}
```

```
// implementation of turnOff()
@override
turnOff() {
  print('MacBook turned off');
}

void main() {
  var macBook = MacBook();
  macBook.turnOn();
  macBook.turnOff();
}
```

[Show Output](#)

Run Online

Info

Note: Most of the time, **abstract class** is used instead of **concrete class** to declare an interface.

Example 2: Interface In Dart

In this example below, there is an abstract class named **Vehicle**. The **Vehicle** class has two abstract methods **start()** and **stop()**. The **Car** class implements the **Vehicle** interface. The **Car** class has to implement the **start()** and **stop()** methods.

```
// abstract class as interface
abstract class Vehicle {
  void start();
  void stop();
}

// implements interface
class Car implements Vehicle {
  @override
  void start() {
    print('Car started');
  }

  @override
  void stop() {
    print('Car stopped');
  }
}
```



```
void main() {  
  var car = Car();  
  car.start();  
  car.stop();  
}
```

[Show Output](#)

Run Online

Multiple Inheritance In Dart

Multiple inheritance means a class can inherit from more than one class. In dart, you can't inherit from more than one class. But you can implement multiple interfaces in a class.

Syntax For Implementing Multiple Interfaces In Dart

```
class ClassName implements Interface1, Interface2, Interface3 {  
  // code  
}
```

Example 3: Interface In Dart With Multiple Interfaces

In this example below, two abstract classes are named **Area** and **Perimeter**. The **Area** class has an abstract method **area()** and the **Perimeter** class has an abstract method **perimeter()**. The **Shape** class implements both the **Area** and **Perimeter** classes. The **Shape** class has to implement the **area()** and **perimeter()** methods.

```
// abstract class as interface  
abstract class Area {  
  void area();  
}  
// abstract class as interface  
abstract class Perimeter {  
  void perimeter();  
}  
// implements multiple interfaces  
class Rectangle implements Area, Perimeter {  
  // properties
```

```

int length, breadth;

// constructor
Rectangle(this.length, this.breadth);

// implementation of area()
@override
void area() {
    print('The area of the rectangle is ${length * breadth}');
}

// implementation of perimeter()
@override
void perimeter() {
    print('The perimeter of the rectangle is ${2 * (length + breadth)}');
}

void main() {
    Rectangle rectangle = Rectangle(10, 20);
    rectangle.area();
    rectangle.perimeter();
}

```

[Show Output](#)

Run Online

Example 4: Interface In Dart

In this example below, there is an abstract class named **Person**. The **Person** class has one property **name** and two abstract methods **run** and **walk**. The **Student** class implements the **Person** interface. The **Student** class has to implement the **run** and **walk** methods.

```

// abstract class as interface
abstract class Person {
    // properties
    String? name;
    // abstract method
    void run();
    void walk();
}

class Student implements Person {
    // properties
    String? name;

    // implementation of run()
    @override
    void run() {
        print('Student is running');
    }
}

```



```
// implementation of walk()
@override
void walk() {
  print('Student is walking');
}

void main() {
  var student = Student();
  student.name = 'John';
  print(student.name);
  student.run();
  student.walk();
}
```

[Show Output](#)

Run Online

Example 5: Interface In Dart

In this example below, there is abstract class named **CalculateTotal** and **CalculateAverage**. The **CalculateTotal** class has an abstract method **total()** and the **CalculateAverage** class has an abstract method **average()**. The **Student** class implements both the **CalculateTotal** and **CalculateAverage** classes. The **Student** class has to implement the **total()** and **average()** methods.

```
// abstract class as interface
abstract class CalculateTotal {
  int total();
}
// abstract class as interface
abstract class CalculateAverage {
  double average();
}
// implements multiple interfaces
class Student implements CalculateTotal, CalculateAverage {
  // properties
  int marks1, marks2, marks3;
  // constructor
  Student(this.marks1, this.marks2, this.marks3);
  // implementation of average()
  @override
  double average() {
    return total() / 3;
  }
  // implementation of total()
  @override
  int total() {
    return marks1 + marks2 + marks3;
  }
}
```

```

    }
}

void main() {
    Student student = Student(90, 80, 70);
    print('Total marks: ${student.total()}');
    print('Average marks: ${student.average()}');
}

```

[Show Output](#)

[Run Online](#)

Difference Between Extends & Implements

extends	implements
Used to inherit a class in another class.	Used to inherit a class as an interface in another class.
Gives complete method definition to sub-class.	Gives abstract method definition to sub-class.
Only one class can be extended.	Multiple classes can be implemented.
It is optional to override the methods.	Concrete class must override the methods of an interface.
Constructors of the superclass is called before the sub-class constructor.	Constructors of the superclass is not called before the sub-class constructor.
The super keyword is used to access the members of the superclass.	Interface members can't be accessed using the super keyword.
Sub-class need not to override the fields of the superclass.	Subclass must override the fields of the interface.

Key Points To Remember

- An interface is a contract that defines the capabilities of a class.
- Dart has no keyword interface, but you can use class or abstract class to declare an interface.

- Use abstract class to declare an interface.
- A class can extend only one class but can implement multiple interfaces.
- Using the interface, you can achieve multiple inheritance in Dart.
- It is used to achieve abstraction.

Video

Watch our video on interface in Dart.